Homework 2 — LLM Chat

Technical Report

Stack: Python, FastAPI, PostgreSQL, Redis, Jinja2, llama-cpp-python

---

1. Frontend & Architecture

Frontend: Two server-rendered HTML pages (`/` and `/chat`). All UI interactions are handled by inline JavaScript — no page reloads after login. Tokens are stored in `localStorage`, and every API call includes `Authorization: Bearer <access_token>`. Expired access tokens are transparently refreshed using the stored refresh token.

Backend: Model-View-Controller (MVC) layered architecture.

Layer	Implementation
Model	PostgreSQL tables via SQLAlchemy async ORM (`app/models/`)
View	`app/templates/index.html`, `app/templates/chat.html` + `static/app.css`
Controller	FastAPI route files (`app/api/auth.py`, `app/api/chats.py`, `app/web/pages.py`)

Each controller calls a single usecase function (`app/services/auth_usecases.py`, `app/services/chats_usecases.py`). Usecases call the store layer (`app/services/users.py`, `app/services/chats.py`) which owns all SQL queries.

---

2. API Endpoints

All `/api/*` routes return JSON. Protected routes require `Authorization: Bearer <access_token>`.

2.1 Authentication (`/api/auth`)

Method	Path	Request	Response
POST	`/register`	`{ login, password }`	`{ id, login }`
POST	`/login`	`{ login, password }`	`{ access_token, refresh_token, token_type }`
POST	`/refresh`	`{ refresh_token }`	`{ access_token, token_type }`

2.2 GitHub OAuth

Method	Path	Behavior
GET	`/auth/github/login`	Returns `{ authorize_url }` to redirect the browser to GitHub
GET	`/auth/github/callback`	Exchanges code → redirects to `/chat?access_token=…&refresh_token=…`

2.3 Chats (`/api/chats`) — protected

Method	Path	Response
GET	`/api/chats`	`ChatOut[]`
POST	`/api/chats`	`ChatOut`
GET	`/api/chats/{chat_id}`	`ChatWithMessagesOut`
POST	`/api/chats/{chat_id}/messages`	`LlmAnswerOut` (user + assistant messages)
DELETE	`/api/chats/{chat_id}`	`{ ok: true }`

2.4 LLM Status

Method	Path	Response
GET	`/api/llm/status`	`{ configured, llama_cpp, model_path, … }`

2.5 Page Routes

Method	Path	Template
GET	`/`	`index.html` (login / register)
GET	`/chat`	`chat.html` (main chat interface)

---

3. Codebase Structure

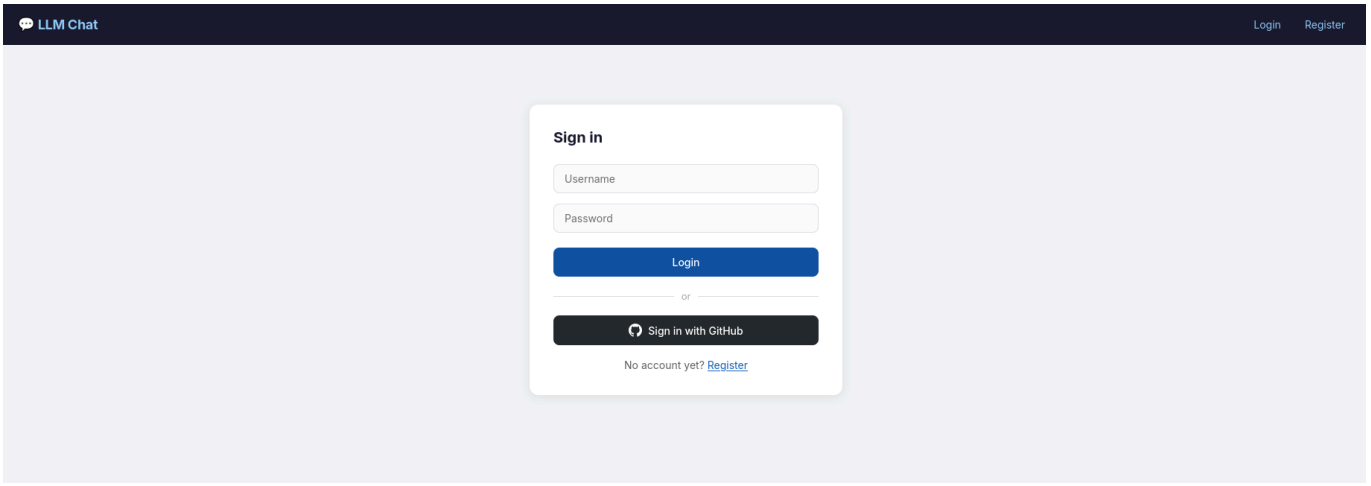
```
app/
├── main.py              # FastAPI app, mounts static files, includes
routers
├── settings.py          # Settings dataclass, reads env vars
├── db.py                # Async SQLAlchemy engine + Base
├── security.py          # bcrypt password hashing, JWT encode/decode
├── redis_client.py      # Async Redis client (lru_cache singleton)
├── schemas.py           # Pydantic request/response models
├── deps.py              # get_current_user (Bearer token → user dict)
├── api/                 # Controllers — thin routes, one usecase call each
│   ├── auth.py
│   ├── chats.py
│   └── llm.py
├── services/            # Business logic + data access
```

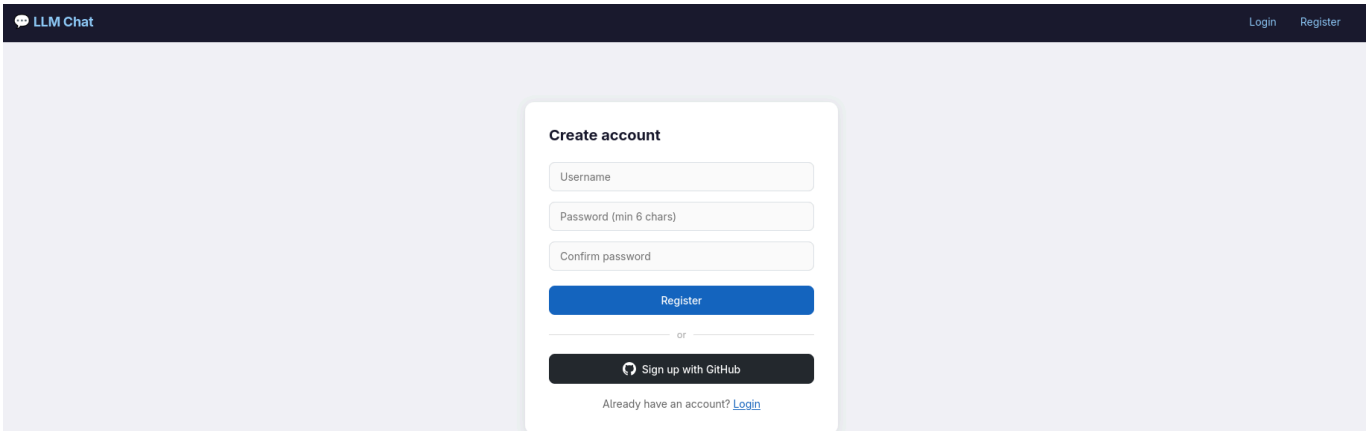
```
│   ├── users.py          # User CRUD (returns plain dicts)
│   ├── auth.py           # issue_tokens, refresh, OAuth state in Redis
│   ├── auth_usecases.py  # register / login / github_callback logic
│   ├── chats.py          # Chat/Message SQL queries (returns plain dicts)
│   ├── chats_usecases.py# Chat business logic
│   ├── llm.py            # llama-cpp-python inference (anyio thread)
│   ├── llm_usecases.py   # LLM status check
│   └── oauth_github.py   # GitHub OAuth HTTP calls
├── web/
│   └── pages.py          # HTML page routes (/ and /chat)
├── templates/
│   ├── index.html        # Login / Register SPA page
│   └── chat.html         # Chat SPA page
└── static/
    └── app.css           # Dark theme CSS

alembic/                  # Database migrations (Alembic)
alembic.ini
requirements.txt
```

---

4. UI Screenshots

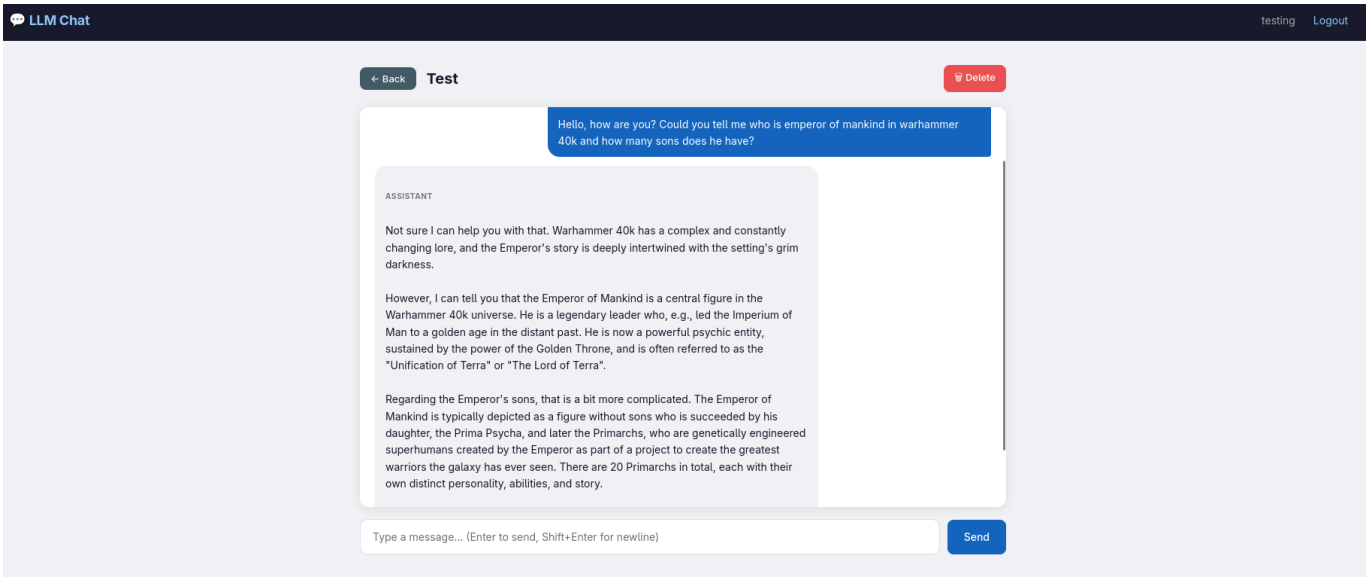
4.1 `/` — Login / Register





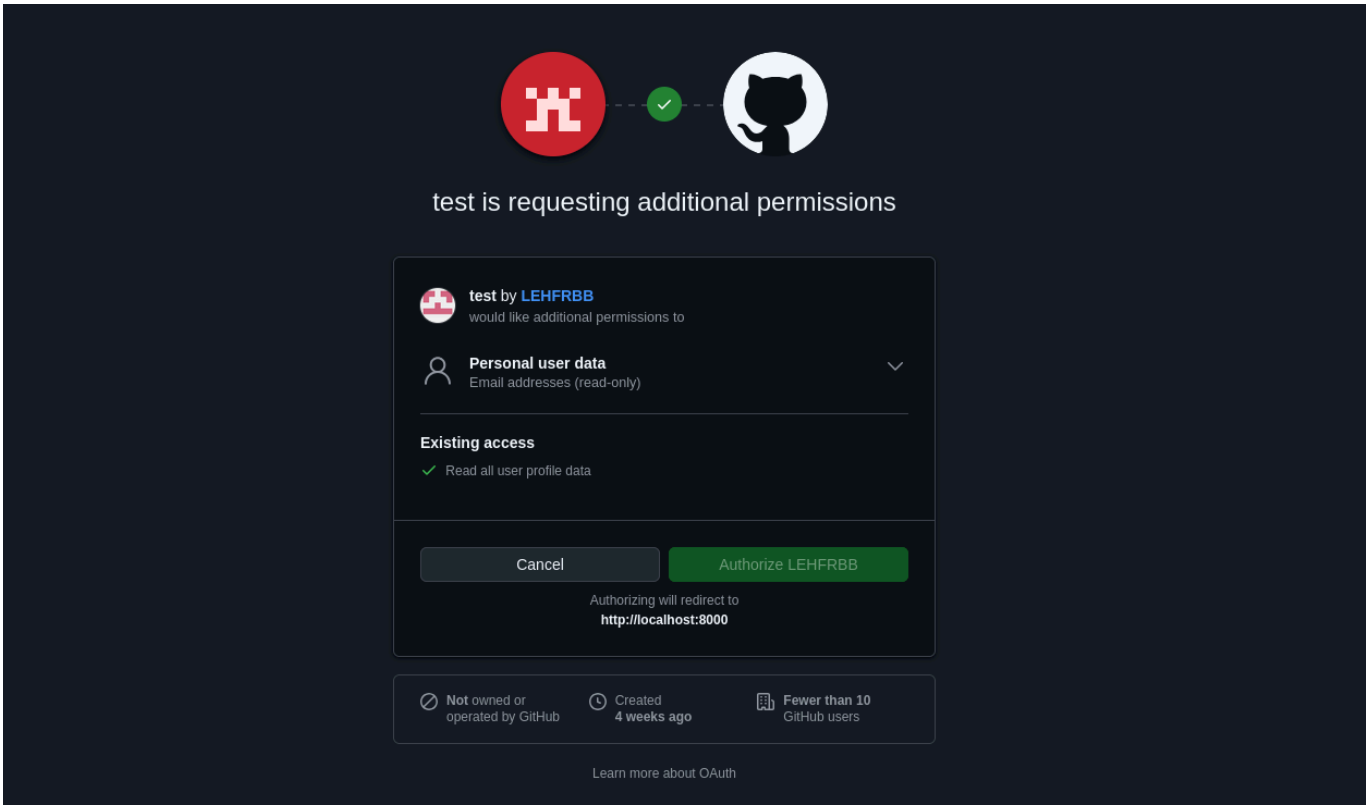
The landing page has two tabs: **Login** and **Register**. A GitHub OAuth button appears in the top right. After successful authentication, tokens are stored in `localStorage` and the user is redirected to `/chat`.

4.2 `/chat` — Main Chat Interface



- **Left sidebar:** brand name, Logout button, "New chat" input + button, list of existing chats (each with a delete × button)
- **Main area:** current chat title in the header, scrollable message history (user messages right-aligned in teal, assistant messages left-aligned in grey), input + Send button at the bottom
- **Sending a message:** the user message appears immediately, a spinning loader shows while waiting for the LLM, then the assistant reply replaces it
- **401 handling:** if the access token expires mid-session, the client automatically calls `/api/auth/refresh` and retries the original request

4.3 GitHub OAuth Flow



Clicking "GitHub OAuth" calls `/auth/github/login` → the server stores a random state in Redis and returns an `authorize_url`. The browser is redirected to GitHub's consent screen. After approval, GitHub redirects back to `/auth/github/callback?code=…&state=…`. The server verifies the state, exchanges the code for a GitHub access token, fetches the user profile, creates or finds the user record, issues JWT+refresh tokens, and redirects the browser to `/chat?access_token=…&refresh_token=…`.

---

5. Database Schema (PostgreSQL)

users

Column	Type	Constraints
id	INTEGER	PK, auto-increment
login	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(255)	nullable (GitHub users have no password)
github_id	VARCHAR(100)	UNIQUE, nullable

chats

Column	Type	Constraints
id	INTEGER	PK, auto-increment
user_id	INTEGER	FK → users.id, NOT NULL
title	VARCHAR(500)	NOT NULL
created_at	DATETIME	default = utcnow

messages

Column	Type	Constraints
id	INTEGER	PK, auto-increment
chat_id	INTEGER	FK → chats.id, NOT NULL
role	VARCHAR(50)	`'user'` or `'assistant'`
content	TEXT	NOT NULL
created_at	DATETIME	default = utcnow

Schema is managed by **Alembic** (`alembic upgrade head` before first run).

---

6. JWT + Refresh Token + Redis

- **Access token:** JWT (HS256), payload `{ sub: user_id, login, iat, exp, type: "access" }`, TTL = 30 min (configurable via `ACCESS_TTL_SECONDS`)

- **Refresh token:** random 32-byte URL-safe string stored in Redis as `refresh:<token> → user_id`, TTL = 30 days (`REFRESH_TTL_SECONDS`)
- **Refresh flow:** client calls `POST /api/auth/refresh` with its refresh token → receives a new access token
- **OAuth state:** stored in Redis as `oauth_state:<state> → "1"`, TTL 10 minutes, deleted after callback
- **Logout:** client clears `localStorage`; no server-side revocation needed (token expires naturally)

---

7. LLM Integration

`app/services/llm.py` provides `answer_chat(messages)` using **llama-cpp-python** with a local `.gguf` model file.

Setting	Environment variable	Default
Model path	`MODEL_PATH`	auto-detected `model.gguf` at project root

- The model is loaded once at first call via `@lru_cache`
- Inference runs in a thread pool via `anyio.to_thread.run_sync` to avoid blocking the async event loop
- The last 12 messages of the current chat are sent as context in `"Role: content"` format
- `GET /api/llm/status` reports whether the model is loaded

---

8. Running the Project

Prerequisites

```
# PostgreSQL running; create database
createdb chatapp

# Redis running
redis-server --daemonize yes

# Python virtual environment
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Environment variables (`.env`)

```
SECRET_KEY=your-random-secret
DATABASE_URL=postgresql://postgres:postgres@localhost/chatapp
REDIS_URL=redis://localhost:6379/0
BASE_URL=http://127.0.0.1:8000

# Optional — GitHub OAuth
```

```
GITHUB_CLIENT_ID=your-client-id
GITHUB_CLIENT_SECRET=your-client-secret

# Optional — local LLM (defaults to model.gguf in project root)
MODEL_PATH=/path/to/model.gguf
```

Apply migrations and run

```
alembic upgrade head
uvicorn app.main:app --reload
```

Open `http://127.0.0.1:8000` in a browser.